SubNetViewer コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

SubNetViewerは、PetriNetのTransitionノードに接続されたサブネット（入力/出力Placeノード）をReact Flowで可視化するコンポーネントです。左ペインにサブネット図、右ペインにプロパティビューアを表示する2ペインレイアウトです。

1.2 使用場面

- PetriNetViewerでTransitionノードをクリックした際のサブネット表示
- ViewRendererDialog内でのサブネット可視化
- Transitionに接続されたPlace（Material/Condition/Result）の関係表示

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時（右ペイン閉じている）

```
┌──────────────────────────────────────────────────────┐
│                    [🔄 ビューリセット] [ℹ️ 情報表示]    │
│                                                      │
│                  React Flow Canvas                   │
│                  (サブネット表示)                      │
```

```
│                                                          │
│   ◯ Material → ⬡ Transition → ◯ Result                  │
│   ◯ Condition ↗                                          │
│                                                          │
│  [MiniMap]                          [Controls]           │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.1.2 プロパティビューア表示時

```
┌──────────────────────────────────┬──────────────────────┐
│  [🔄] [ℹ️]                       │ [ノード名]           │
│                                  │ プロパティ    [×]    │
│        React Flow Canvas         ├──────────────────────┤
│       （サブネット表示）          │ [基本情報]           │
│                                  │ [テンプレート]       │
│  ◯ Material → ⬡ → ◯ Result       │ [データ]             │
│  ◯ Condition ↗                   │                      │
│                                  │ ID: place_001        │
│  [MiniMap]     [Controls]        │ 名前: サンプル材料   │
│                                  │                      │
└──────────────────────────────────┴──────────────────────┘
    60%                                   40%
```

2.2 主要UI要素

2.2.1 ツールバー（左ペイン右上）

- **背景**: 半透明白（rgba(255, 255, 255, 0.9)）
- **角丸**: token.borderRadius
- **シャドウ**: token.boxShadow
- **ボタン**:
  - ビューリセット: ReloadOutlined アイコン
  - 情報表示: InfoCircleOutlined アイコン

2.2.2 左ペイン（React Flow）

- **幅**: 右ペイン閉じ時は100%、開いている時は60%

- **トランジション**: 0.3s ease
- **React Flow設定**:
  - fitView: 有効
  - fitViewOptions: { padding: 0.2, minZoom: 0.5, maxZoom: 1.5 }
  - MiniMap: 100x150px
  - Controls: ズーム、フィット等のコントロール
  - Background: グリッドパターン（#aaa、gap: 16）

2.2.3 右ペイン（PropertyViewer）

- **幅**: 40%
- **ボーダー**: 左側にボーダー
- **背景**: token.colorBgContainer
- **コンテンツ**: PropertyViewerコンポーネント

---

3. 機能仕様

3.1 サブネットデータの抽出

3.1.1 処理フロー

```
useEffect(() => {
  if (!nodeName || !open) {
    return;
  }

  const fetchSubnetData = async () => {
    try {
      setLoading(true);
      setError(null);

      // 複数method対応
      let currentArcs = petriNetData.arcs;
      let currentPlaces = petriNetData.places;
```

```
    if (multiMethodMode && arcsData && placesData) {
      currentArcs = arcsData;
      currentPlaces = placesData;
    }

    // トランジションに接続されたアークをフィルタリング
    const transitionArcs = currentArcs.filter((arc) =>
      arc.source === nodeName || arc.target === nodeName
    );

    if (transitionArcs.length === 0) {
      setError(`トランジション "${nodeName}" に接続されたアークがありません`);
      setLoading(false);
      return;
    }

    // arcから関連するplaceIDを収集
    const relatedPlaceIds = new Set<string>();
    const relatedArcs: ArcInfo[] = [];

    transitionArcs.forEach((arc) => {
      relatedArcs.push({
        id: arc.id,
        source: arc.source,
        target: arc.target,
        weight: '1',
        description: arc.id,
      });

      if (arc.source !== nodeName) {
        relatedPlaceIds.add(arc.source);
      }
      if (arc.target !== nodeName) {
        relatedPlaceIds.add(arc.target);
      }
    });

    // placesデータを使用
    const relatedPlaces: PlaceInfo[] = petriNetData.places
      .filter((place) => relatedPlaceIds.has(place.id))
      .map((place) => {
        // ... テンプレート情報の取得 ...
      });

    // サブネットデータを構築
    const subnet: SubnetData = {
      transition: { id: nodeName, name: nodeName, type: 'normal' },
      places: relatedPlaces,
      arcs: relatedArcs,
    };
```

```
      setSubnetData(subnet);

      // React Flow形式に変換
      const { nodes: flowNodes, edges: flowEdges } = convertSubnetToReactFlow(subnet);

      setNodes(flowNodes);
      setEdges(flowEdges);
      setLoading(false);

    } catch (err) {
      console.error('❌ [SubNetViewer] サブネットデータ構築エラー:', err);
      setError('サブネットデータの構築中にエラーが発生しました');
      setLoading(false);
    }
  };

  fetchSubnetData();
}, [maimlData, nodeName, templateMap, petriNetData, setNodes, setEdges, open, multiMethodMode,
arcsData, placesData]);
```

3.1.2 UI動作

- **開始時**: loadingフラグをtrueに設定、Spinner表示
- **アーク抽出**: nodeName（トランジション）に接続されたアークをフィルタリング
- **Place収集**: アークから関連するPlaceIDを収集
- **テンプレート情報**: templateMapからテンプレート情報を取得
- **変換**: convertSubnetToReactFlowでReact Flow形式に変換
- **成功時**: nodes、edgesをStateに設定、Canvas表示
- **エラー時**: errorメッセージを設定、Alert表示

3.2 ノードクリック処理

3.2.1 処理フロー

```
const handleNodeClick: NodeMouseHandler = useCallback((event, node) => {
  setSelectedNodeId(node.id);
  setSelectedNode(node);

  // Placeノードかつ Templateの場合
```

```
  if (node.data.nodeType === 'place' && node.data.isTemplate && node.data.template) {
    const extractedProps = extractTemplateProperties(
      node.data.template.element,
      node.data.templateType as string
    );

    // 構造化プロパティに変換
    const basicProperties: Record<string, PropertyObject> = {
      id: { key: 'id', value: extractedProps.id, type: 'string', description: 'テンプレートID'
},
      type: { key: 'type', value: extractedProps.type, type: 'string', description: 'テンプレー
ト種別' },
      name: { key: 'name', value: extractedProps.name, type: 'string', description: 'テンプレー
ト名' },
    };

    const templateProperties: Record<string, PropertyObject> = {};
    Object.entries(extractedProps).forEach(([key, value]) => {
      if (!['id', 'type', 'name'].includes(key)) {
        templateProperties[key] = {
          key,
          value: value ?? '値なし',
          type: typeof value,
          description: key,
        };
      }
    });

    const dataProperties: Record<string, PropertyObject> = {
      coordinates: {
        key: 'coordinates',
        value: `(${node.position.x.toFixed(2)}, ${node.position.y.toFixed(2)})`,
        type: 'coordinates',
        description: 'ノード座標',
      },
    };

    setStructuredProperties({
      basic: basicProperties,
      template: templateProperties,
      data: dataProperties,
    });
  }

  // 右ペインを開く
  setRightPanelOpen(true);
}, [extractTemplateProperties]);
```

- **Placeノード**: extractTemplatePropertiesでプロパティ抽出 → 右ペイン表示

- **Transitionノード**: 基本情報のみ表示 → 右ペイン表示

3.3 テンプレートプロパティ抽出

3.3.1 処理フロー

```
const extractTemplateProperties = useCallback((templateElement: Element | null, templateType:
string) => {
  if (!templateElement) {
    return { id: 'unknown', type: templateType, name: '不明' };
  }

  const properties: Record<string, unknown> = {
    id: templateElement.getAttribute('id'),
    type: templateType.charAt(0).toUpperCase() + templateType.slice(1),
    name: templateElement.getAttribute('name') || '名称未設定',
  };

  // 属性を追加
  for (let i = 0; i < templateElement.attributes.length; i++) {
    const attr = templateElement.attributes[i];
    if (!['id', 'name'].includes(attr.name)) {
      properties[attr.name] = attr.value;
    }
  }

  // 子要素をプロパティに追加
  for (let i = 0; i < templateElement.children.length; i++) {
    const child = templateElement.children[i];
    const tagName = child.tagName;

    let value: unknown = child.textContent?.trim() || '';

    // 数値変換
    if (typeof value === 'string' && /^-?\d+(\.\d+)?$/.test(value)) {
      value = parseFloat(value);
    }

    // Boolean変換
    if (value === 'true') value = true;
    if (value === 'false') value = false;

    // 同じタグ名が複数ある場合は配列化
    if (properties[tagName]) {
      if (!Array.isArray(properties[tagName])) {
        properties[tagName] = [properties[tagName]];
```

```
      }
      (properties[tagName] as unknown[]).push(value);
    } else {
      properties[tagName] = value;
    }
  }

  return properties;
}, []);
```

- **属性抽出**: Element.attributesから属性を取得
- **子要素抽出**: Element.childrenから子要素を取得
- **型変換**: 数値・Boolean自動変換
- **配列化**: 同名タグは配列として管理

3.4 その他の機能

3.4.1 ビューリセット

```
const handleResetView = useCallback(() => {
  setRightPanelOpen(false);
  setSelectedNode(null);
}, []);
```

- 右ペインを閉じる
- 選択ノードをクリア

3.4.2 情報表示

```
const handleShowInfo = useCallback(() => {
  alert(`サブネット情報\n\n中心ノード: ${nodeName}\n関連プレース数: ${subnetData?.places.length
|| 0}\nアーク数: ${subnetData?.arcs.length || 0}\nテンプレート参照:
${Object.keys(templateMap).length}件`);
}, [nodeName, subnetData, templateMap]);
```

- 簡易的なalertでサブネット情報を表示

---

4. データ仕様

4.1 Props定義

```
interface SubNetViewerProps {
  open: boolean;
  nodeName: string;
  maimlData: string;
  templateMap: Record<string, { element: Element }>;
  petriNetData: {
    places: Array<{
      id: string;
      data: {
        label?: string;
        stereotype?: string;
        template?: { id: string };
        templateId?: string;
      };
    }>;
    arcs: Array<{
      id: string;
      source: string;
      target: string;
    }>;
  };
  onClose: () => void;
  arcsData?: Array<...> | null;
  placesData?: Array<...> | null;
  multiMethodMode?: boolean;
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	表示状態
nodeName	string	✅	TransitionノードのID
maimlData	string	✅	MAIML XML文字列

templateMap	Record<string, { element: Element }>	✅	テンプレートマップ
petriNetData	Object	✅	Petri Net構造データ
onClose	Function	✅	閉じる時のコールバック
arcsData	Array	-	複数method対応用アークデータ
placesData	Array	-	複数method対応用Placeデータ
multiMethodMode	boolean	-	複数methodモードフラグ

4.2 State定義

```
const [nodes, setNodes, onNodesChange] = useNodesState<Node>([]);
const [edges, setEdges, onEdgesChange] = useEdgesState<Edge>([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
const [subnetData, setSubnetData] = useState<SubnetData | null>(null);
const [rightPanelOpen, setRightPanelOpen] = useState(false);
const [selectedNode, setSelectedNode] = useState<Node | null>(null);
const [selectedNodeId, setSelectedNodeId] = useState<string | null>(null);
const [structuredProperties, setStructuredProperties] = useState<{
  basic: Record<string, PropertyObject>;
  template: Record<string, PropertyObject>;
  data: Record<string, PropertyObject>;
}>({ basic: {}, template: {}, data: {} });
```

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しなし

備考:

- SubNetViewerはPropsとして渡されたデータを使用
- MAIML XMLの解析はコンポーネント内で実施

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
アークが見つからない	「トランジション "[nodeName]" に接続されたアークがありません」Alert表示
サブネットデータ構築エラー	「サブネットデータの構築中にエラーが発生しました」Alert表示
テンプレート要素なし	デフォルトプロパティ（id: 'unknown', name: '不明'）を返す

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/dialogs/SubNetViewer.tsx`

7.2 関連ファイル

- **ユーティリティ**: `frontend/src/utils/subnetReactFlowConverter.ts`
- **プロパティビューア**: `frontend/src/components/viewer/dialogs/PropertyViewer.tsx`
- **カスタムノード**:
  - `frontend/src/components/nodes/MaterialTemplateNode.tsx`
  - `frontend/src/components/nodes/ConditionTemplateNode.tsx`
  - `frontend/src/components/nodes/ResultTemplateNode.tsx`
  - `frontend/src/components/nodes/TransitionNode.tsx`

7.3 依存ライブラリ

- `reactflow` : ReactFlow、MiniMap、Controls、Background、useNodesState、useEdgesState、ReactFlowProvider
- `antd` : Button, Space, Flex, Spin, Alert, Typography, Tooltip, theme
- `@ant-design/icons` : InfoCircleOutlined, ReloadOutlined

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline